

DOCUMENTO TÉCNICO

APLICACIÓN DE GESTIÓN CLUB POLIDEPORTIVO

1. Introducción

1.1 Objetivo del proyecto

Este proyecto consiste en el desarrollo de una aplicación de escritorio orientada a la gestión integral de un club polideportivo. La aplicación ha sido desarrollada en C# utilizando WPF como tecnología de interfaz gráfica y está destinada al personal administrativo del club.

El objetivo principal de la aplicación es centralizar y simplificar todas las tareas habituales de gestión, permitiendo controlar de forma eficiente los socios, las instalaciones deportivas, las cuotas, las reservas y la generación de informes.

La aplicación permite:

- Gestionar socios y sus datos personales.
- Organizar las instalaciones por categorías.
- Registrar y controlar cuotas anuales o mensuales.
- Gestionar reservas de instalaciones evitando solapamientos horarios.
- Generar informes estadísticos y operativos mediante Crystal Reports.

El sistema no incorpora autenticación de usuarios, ya que está pensado para utilizarse en equipos internos del club con acceso restringido únicamente al personal autorizado.

1.2 Contexto de uso

La aplicación está diseñada para ser utilizada desde un único entorno de administración dentro del club deportivo. El personal autorizado accede directamente al sistema desde los equipos de trabajo del propio club, por lo que el control de acceso se delega al sistema operativo y al entorno físico donde se ejecuta la aplicación.

Todas las funcionalidades han sido pensadas para un uso sencillo e intuitivo, permitiendo realizar las operaciones diarias de gestión de forma rápida y centralizada.

2. Tecnologías utilizadas

La aplicación ha sido desarrollada utilizando diferentes tecnologías y herramientas, cada una de ellas seleccionada por su estabilidad, compatibilidad y facilidad de integración dentro del ecosistema .NET.

C# 7.3

Es el lenguaje principal de desarrollo de la aplicación. Se eligió por su robustez, tipado estático y excelente integración con Visual Studio y el framework .NET.

.NET Framework 4.8

Se utiliza como plataforma de ejecución principal. Esta versión ofrece una gran estabilidad y compatibilidad con tecnologías utilizadas en el proyecto como Entity Framework 6 y Crystal Reports.

WPF (Windows Presentation Foundation)

Es el framework utilizado para construir toda la interfaz gráfica de usuario. Permite separar claramente el diseño visual realizado en XAML de la lógica de programación escrita en C#.

Además, WPF facilita:

- El uso de estilos globales.
- La reutilización de componentes visuales.
- El binding de datos.
- La creación de interfaces modernas y dinámicas.

Entity Framework 6

Se utiliza como ORM (Object Relational Mapper) para trabajar con la base de datos mediante objetos C# en lugar de escribir consultas SQL manualmente.

Gracias a Entity Framework se consigue:

- Reducir código repetitivo.
- Centralizar el acceso a datos.
- Mejorar la mantenibilidad del proyecto.
- Facilitar el trabajo con relaciones entre entidades.

SQL Server Express

Es el sistema gestor de base de datos utilizado para almacenar toda la información del club de forma relacional y segura.

La base de datos mantiene la integridad de los datos mediante claves primarias, claves foráneas y restricciones de unicidad.

Crystal Reports

Se utiliza para la generación de informes profesionales integrados directamente en la aplicación.

Permite crear informes complejos con:

- Agrupaciones.
- Totales automáticos.
- Diseños personalizados.
- Filtros dinámicos.

Git y GitHub

Se utilizan como sistema de control de versiones del proyecto.

Gracias a Git y GitHub es posible:

- Mantener un historial de cambios.
- Trabajar con distintas versiones del proyecto.
- Gestionar ramas de desarrollo.
- Disponer de una copia remota del código fuente.

MSTest

Es el framework de pruebas automatizadas de Microsoft integrado en Visual Studio. Se utiliza para definir y ejecutar los tests unitarios y de integración del proyecto. Se eligió por su integración nativa con el ecosistema .NET y Visual Studio, sin necesidad de instalar herramientas externas.

Inteligencia artificial

He utilizado varias herramientas de IA en este proyecto (copilot, chatGPT) para las siguientes operaciones:

- Generación scripts de datos de manera ágil
- Generar esquemas para los documentos
- Resolución de dudas generales y sobre mejores prácticas
- Resolución de errores
- Sugerencias de mejoras
- Dudas sobre como realizar algunas operaciones (C#)

- Y seguro que alguna cosa mas que se me olvida mencionar

3. Arquitectura de la aplicación

3.1 Arquitectura en capas basada en MVC

La aplicación sigue una arquitectura organizada en capas basada en el patrón MVC. Cada capa tiene una responsabilidad específica y se encuentra separada físicamente en proyectos distintos dentro de la solución de Visual Studio.

Esta separación mejora:

- El mantenimiento del código.
- La escalabilidad del sistema.
- La reutilización de componentes.
- La organización general del proyecto.

Las capas principales son las siguientes:

Capa Vista (View)

Contiene todas las ventanas, controles y elementos visuales desarrollados con WPF.

Es la única capa con la que interactúa directamente el usuario. Su responsabilidad se limita exclusivamente a mostrar información y recoger acciones del usuario.

Las vistas nunca acceden directamente a la base de datos.

Capa Controller (Controller)

Contiene la lógica de negocio principal de la aplicación.

Los controladores reciben las peticiones realizadas desde las vistas, ejecutan las validaciones necesarias y coordinan las operaciones correspondientes.

Cada método devuelve un valor booleano indicando si la operación se ha realizado correctamente o si se ha producido algún error.

Capa API (Controller)

Actúa como intermediaria entre los controladores y los repositorios.

Su función principal es centralizar operaciones reutilizables y encapsular posibles excepciones provenientes del acceso a datos.

Capa Repository (model)

Es la única capa que interactúa directamente con Entity Framework y con SQL Server.

Su responsabilidad se limita a realizar operaciones CRUD y consultas sobre la base de datos.

No contiene lógica de negocio.

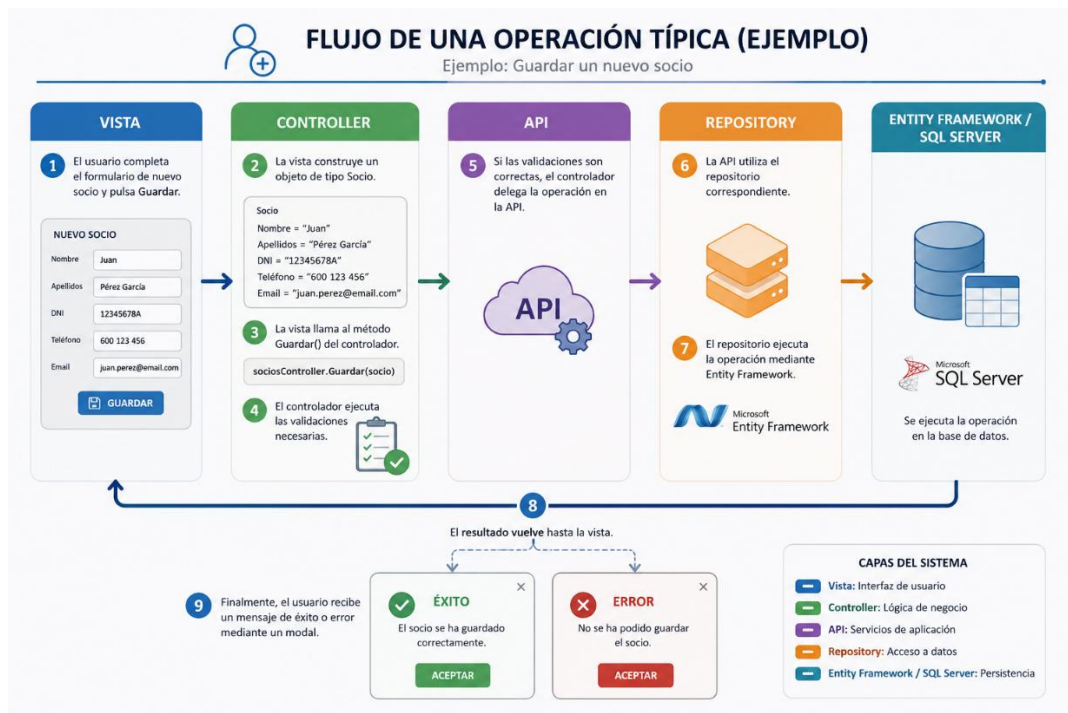
Entity Framework y SQL Server (model)

Entity Framework actúa como capa de abstracción entre la aplicación y SQL Server, traduciendo automáticamente las operaciones realizadas sobre objetos C# a sentencias SQL.

3.2 Flujo de una operación típica (ejemplo)

Cuando el usuario guarda un nuevo socio desde la aplicación, el flujo sigue varios pasos:

1. La vista recoge los datos introducidos en el formulario.
2. Se construye un objeto de tipo Socio.
3. La vista llama al método correspondiente del controlador.
4. El controlador ejecuta las validaciones necesarias.
5. Si las validaciones son correctas, el controlador delega la operación en la API.
6. La API utiliza el repositorio correspondiente.
7. El repositorio ejecuta la operación mediante Entity Framework.
8. El resultado vuelve hasta la vista.
9. Finalmente, el usuario recibe un mensaje de éxito o error mediante un modal.



3.3 Regla de arquitectura principal

Una de las normas fundamentales del proyecto es que las vistas nunca pueden acceder directamente a APIs ni repositorios.

Toda operación debe pasar obligatoriamente por el controlador correspondiente.

Esta regla permite:

- Mantener una arquitectura limpia.
- Garantizar la ejecución de las validaciones.
- Centralizar el control de errores.
- Evitar lógica de negocio en la interfaz gráfica.

4. Estructura de la solución

La solución de Visual Studio está organizada en varios proyectos independientes, cada uno con una responsabilidad concreta.

Común

Biblioteca de clases compartida entre todos los proyectos.

Incluye:

- Métodos de validación.
- Utilidades generales.

- Clases para modales.
- Configuración global del club.
- Conversores para WPF.

ProyectoFinal.model_new

Contiene:

- Entidades generadas por Entity Framework.
- El DbContext principal.
- Repositorios.
- Extensiones parciales de entidades.

Toda la lógica relacionada con el modelo de datos se centraliza en este proyecto.

ProyectoFinal.controller_new

Contiene:

- Controladores.
- APIs internas.
- Validaciones de negocio.

Actúa como núcleo principal de la lógica de la aplicación.

ProyectoFinal.view_new

Es el proyecto principal WPF ejecutable.

Incluye:

- MainWindow.
- UserControls de cada módulo.
- Ventanas modales.
- Estilos globales.
- Recursos visuales.

ProyectoFinal.Reports_new

Contiene:

- Informes Crystal Reports.
- DataSets tipados.

- Clases Builder encargadas de preparar los datos.

Centra todo lo relacionado con los informes salvo la visualización que requiere de un proyecto adicional que permita utilizar el visor de informes.

ProyectoFinal.ReportsView_new

Proyecto auxiliar encargado únicamente de mostrar los informes mediante el visor Crystal Reports integrado.

ProyectoFinal.test

Proyecto destinado a pruebas automatizadas mediante MSTest.

Incluye:

- Tests unitarios.
- Tests de validación.
- Tests de integración con base de datos.

5. Base de datos

5.1 Sistema gestor

La aplicación utiliza SQL Server Express como motor de base de datos principal.

La base de datos recibe el nombre de:

ClubPolideportivoDB

El acceso se realiza mediante Entity Framework y el proveedor System.Data.SqlClient.

5.2 Estructura de tablas

La base de datos está compuesta por varias tablas principales:

Socios

Almacena la información personal de los socios del club.

Incluye:

- Nombre.
- Apellidos.

- DNI.
- Teléfono.
- Email.
- Fecha de alta.
- Estado activo/inactivo.

Tipos Instalacion

Contiene las categorías de instalaciones deportivas, como:

- Gimnasio.
- Piscina.
- Pádel.
- Tenis.

Instalaciones

Almacena cada instalación física disponible en el club junto con:

- Tipo de instalación.
- Precio por hora.
- Disponibilidad.
- Observaciones.

Cuotas

Registra las cuotas asociadas a cada socio.

Permite controlar:

- Pagos realizados.
- Cuotas pendientes.
- Fechas de vencimiento.

Reservas

Contiene todas las reservas realizadas sobre las instalaciones.

Incluye:

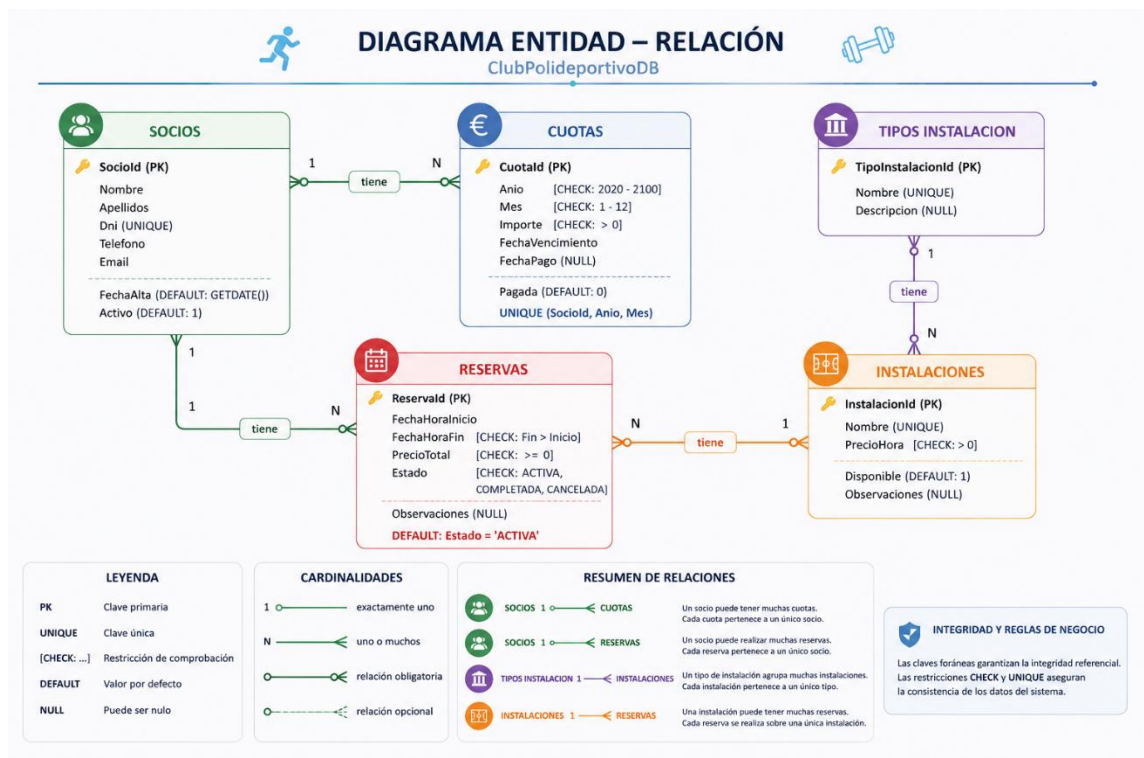
- Socio asociado.
- Instalación reservada.
- Fecha y hora.

- Precio total.
- Estado.

5.3 Relaciones principales

El modelo relacional implementa las siguientes relaciones:

- Un socio puede tener múltiples cuotas.
- Un socio puede tener múltiples reservas.
- Una instalación pertenece a un tipo de instalación.
- Una instalación puede tener muchas reservas.
- Una reserva puede tener varios pagos asociados.



5.4 Reglas de integridad

La base de datos implementa distintas restricciones para proteger la consistencia de los datos.

Por ejemplo:

- No puede existir una reserva sin socio válido.
- No puede existir una reserva sin instalación válida.

- El DNI de los socios es único.

6. Entity Framework — Database First

6.1 Enfoque utilizado

Se ha utilizado el enfoque Database First.

Este enfoque consiste en:

1. Diseñar primero la base de datos.
2. Generar automáticamente las entidades C# desde SQL Server.

Es una opción adecuada cuando se diseña previamente el modelo de datos.

6.2 Fichero EDMX

El fichero principal de Entity Framework es:

ClubPolideportivoDB.edmx

Este fichero contiene:

- El modelo conceptual.
- El modelo físico de la base de datos.
- El mapeo entre ambos.

Visual Studio genera automáticamente las entidades y el DbContext a partir de este fichero.

6.3 DbContext y conexión

El contexto principal generado por Entity Framework se denomina:

ClubPolideportivoDBEntities

La cadena de conexión se almacena en el fichero App.config de los proyectos ejecutables.

Una anotación importante es que la cadena de conexión en un archivo con el nombre mencionado también debe existir en el proyecto inicial que es la vista, si no, no funciona. Por mucho que el modelo y los repositorios tengan acceso a la cadena de conexión, si no se hace así, no hay manera posible de tener acceso a datos.

6.4 Clase base de repositorios

Todos los repositorios heredan de una clase base común llamada RepositoryBase.

Esto permite:

- Centralizar la configuración.
- Reutilizar código.
- Mantener coherencia en el acceso a datos.

Es una manera de no tener que definir la conexión a BBDD en cada uno de los repositorios. Si en un futuro se tuviera que cambiar o añadir una nueva, solo se tendría que cambiar en un archivo

6.5 Extensiones parciales

Las entidades generadas automáticamente no deben modificarse directamente.

Por ello, las propiedades adicionales se implementan mediante clases parciales separadas.

Un ejemplo es la ampliación de la clase socios donde podemos encontrar la propiedad calculada "NombreCompleto", que utilizaremos en varias partes de la aplicación

6.6 Ventajas del uso de Entity Framework

El uso de Entity Framework aporta múltiples ventajas:

- Reduce la necesidad de escribir SQL manual.
- Facilita el trabajo con relaciones.
- Simplifica consultas mediante LINQ.
- Gestiona automáticamente conexiones y cambios de entidades.
- Mejora la legibilidad del código.

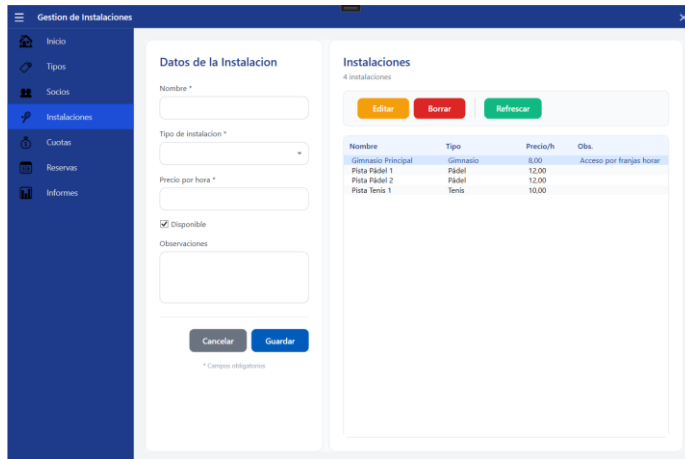
7. Diseño de interfaz — WPF

7.1 Estructura general

La aplicación utiliza una única ventana principal llamada MainWindow.

Esta ventana está dividida en:

- Menú lateral de navegación.
- Zona principal de contenido dinámico.
- Cabecera con título y botón de cerrar



7.2 Menú lateral colapsable

El menú lateral puede expandirse o contraerse mediante un botón tipo hamburguesa.

Cuando está expandido:

- Muestra iconos y texto.

Cuando está contraído:

- Muestra únicamente los iconos.

Cada módulo dispone de un indicador visual que muestra cuál está activo. Y se puede pulsar independientemente de si el menú está colapsado o no.

7.3 Estilos globales

Todos los estilos visuales se encuentran centralizados en el fichero:

Styles.xaml

Esto garantiza una interfaz uniforme y facilita el mantenimiento visual del proyecto. O lo que es lo mismo, permite unificar los estilos de los objetos utilizados en toda la aplicación, optimizando código. Por ejemplo, todos los inputs y botones tienen los mismos estilos y solo están definidos en dicho archivo. En el ejemplo que se adjunta se puede apreciar como a un combobox normal se le aplica el estilo del ComboBoxRedondeado que está definido en el archivo de estilos

```
XAML
1 <ComboBox x:Name="cmbFiltroSocio"
2     Style="{StaticResource ComboBoxRedondeado}"
3     Width="220"
4     Margin="0,0,16,0"
5     DisplayMemberPath="NombreCompleto"
6     SelectedValuePath="SocioId"
7     SelectionChanged="CmbFiltroSocio_SelectionChanged"/>
```

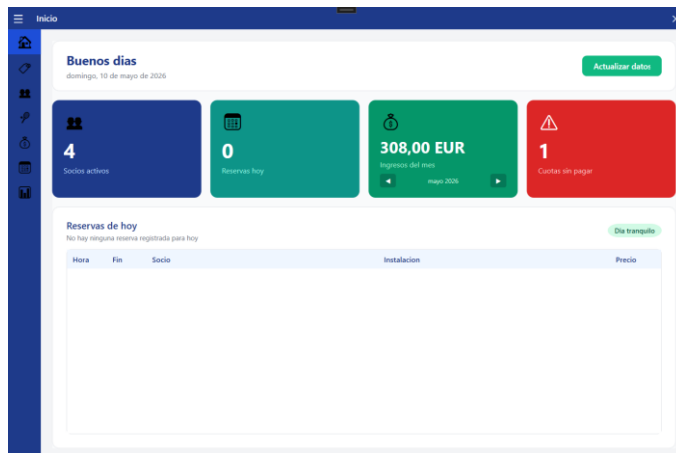
7.4 Vista de inicio

La vista principal muestra información resumida del estado actual del club.

Incluye:

- Número de socios activos.
- Instalaciones disponibles.
- Reservas del día.
- Ingresos diarios.

También incorpora una tabla con las reservas del día de hoy.



7.5 Vistas de gestión

Todos los módulos siguen una estructura visual similar:

- Zona superior de filtros.
- DataGrid como elemento principal.
- Formulario lateral o inferior, dependiendo de la cantidad de datos que se requieran

Esto mejora la experiencia de uso y mantiene consistencia visual entre pantallas.

Tipos de Instalacion

Datos del Tipo de Instalacion

Nombre *

Descripcion

* Campos obligatorios

Tipos de Instalacion

3 tipos

Nombre	Descripcion
Gimnasio	Sala de musculación y fitness
Pádel	Pistas de pádel del club
Tenis	Pistas de tenis del club

Gestion de Socios

Socios Registrados

4 socios

Nombre	Apellidos	DNI	Email	Telefono	Fecha Alta
Prueba	duffiduff	scldsduff	prueba@prueba.es	686022962	10/04/2026
Maria	Juanes Ripoll	12345	maria@maria.es	666222333	05/05/2026
Pedro	Martinez Macdon	457654321	Pedro@prueba.es	666111222	04/05/2026
Juan	Ruiz	asdd	juan@juan.es		10/04/2026

Datos del Socio

Nombre *

Apellidos *

DNI *

Email *

Telefono

☐ Activo

* Campos obligatorios

8. Módulos funcionales implementados

8.1 Gestión de tipos de instalación

Permite crear, editar y eliminar categorías de instalaciones.

El sistema evita eliminar tipos que estén siendo utilizados por instalaciones activas.

8.2 Gestión de socios

Permite:

- Registrar socios.
- Modificar datos.
- Realizar bajas lógicas.
- Buscar socios en tiempo real.

El sistema valida automáticamente:

- Formato de email.
- Formato de teléfono.
- Duplicados de DNI.

8.3 Gestión de instalaciones

Permite gestionar y buscar las instalaciones físicas del club, además de controlar su disponibilidad.

8.4 Gestión de cuotas

Permite registrar y controlar pagos de cuotas.

El sistema impide duplicados para un mismo socio y periodo.

8.5 Gestión de reservas

Es el módulo más importante y completo del sistema. Permite la gestión general de las reservas.

Incluye:

- Validación de horarios.
- Detección de solapamientos.
- Cálculo automático del precio.
- Reserva asistida. Selección visual de franjas horarias (huecos disponibles).
- Validación de usuarios activos e instalaciones disponibles

Nueva Reserva Asistida

Huecos libres del domingo, 10 mayo 2026

Fecha: 10/05/2026

Instalacion: (Todas las instalaciones)

84 huecos disponibles

Instalacion	Tipo	Disponible de	a	Duracion	Precio/h
Gimnasio Principal	Gimnasio	08:00	08:30	30min	8,00 EUR
Gimnasio Principal	Gimnasio	08:30	09:00	30min	8,00 EUR
Gimnasio Principal	Gimnasio	09:00	09:30	30min	8,00 EUR
Gimnasio Principal	Gimnasio	09:30	10:00	30min	8,00 EUR
Gimnasio Principal	Gimnasio	10:00	10:30	30min	8,00 EUR
Gimnasio Principal	Gimnasio	10:30	11:00	30min	8,00 EUR
Gimnasio Principal	Gimnasio	11:00	11:30	30min	8,00 EUR
Gimnasio Principal	Gimnasio	11:30	12:00	30min	8,00 EUR

Hora inicio: 08:00

Hora fin: 09:30

Precio estimado: 12,00 EUR

3 hueco(s) seleccionado(s) - Gimnasio Principal - 90 min en total

Cancelar

Usar este hueco

8.6 Vista de informes

La aplicación incorpora una vista específica destinada a la generación y consulta de informes mediante Crystal Reports. Estos informes permiten al personal administrativo obtener información detallada sobre la actividad del club, facilitando el control operativo y el análisis de datos.

Cada informe se genera dinámicamente a partir de la información almacenada en la base de datos y se muestra en una ventana independiente utilizando el visor integrado de Crystal Reports para WPF.

Informe Socios Activos

El informe de socios activos permite obtener un listado completo de todos los socios que actualmente se encuentran dados de alta en el club. Su objetivo principal es facilitar al personal administrativo una consulta rápida y clara de los miembros activos.

El informe utiliza exclusivamente la tabla Socios, aplicando un filtro interno sobre el campo Activo para mostrar únicamente aquellos registros cuyo valor es verdadero.

No dispone de filtros adicionales, ya que siempre muestra el conjunto completo de socios activos en el momento de generar el informe.

La información se presenta en forma de listado simple, sin agrupaciones, incluyendo datos como:

- Nombre
- Apellidos
- DNI
- Teléfono
- Email
- Fecha de alta

Crystal Reports permite además mostrar un contador total de socios activos en la cabecera del informe.

Desde el punto de vista técnico, el informe es construido por la clase SociosActivosReportBuilder, que obtiene los datos mediante SociosAPI.ObtenerTodos(soloActivos: true). Posteriormente, los datos se cargan en el DataSet tipado SociosActivosDataSet y se devuelve un objeto ReportDocument de tipo InformeSociosActivos, listo para mostrarse en el visor de informes.

Informe Reservas por Instalación

El informe de reservas por instalación permite analizar el nivel de ocupación y uso de las distintas instalaciones deportivas del club.

Su finalidad es identificar cuáles son las instalaciones más utilizadas y conocer los ingresos generados por cada una de ellas.

El informe utiliza información de la tabla Reservas, junto con los datos relacionados de Instalaciones y Socios.

No aplica filtros de fecha, por lo que muestra todas las reservas almacenadas en la base de datos.

Las reservas se ordenan por nombre de instalación y posteriormente por fecha de inicio, permitiendo que Crystal Reports agrupe automáticamente la información por instalación.

Dentro de cada grupo se muestran datos como:

- Nombre de la instalación
- Nombre del socio
- Fecha y hora de inicio
- Fecha y hora de fin
- Precio total
- Estado de la reserva

Además, Crystal Reports calcula automáticamente:

- Número total de reservas por instalación
- Subtotal de ingresos generados

La generación técnica del informe se realiza mediante la clase ReservasPorInstalacionReportBuilder, que obtiene todos los registros utilizando ReservasAPI.ObtenerTodos(). Los datos se ordenan mediante LINQ, se cargan en el DataSet tipado ReservasPorInstalacionDataSet y finalmente se devuelve el ReportDocument correspondiente.

Informe Ingresos por Tipo de Instalación

Este informe ofrece una visión económica general del club agrupando la información por tipo de instalación deportiva.

Permite comparar qué categorías generan un mayor volumen de reservas e ingresos, como por ejemplo:

- Piscina

- Gimnasio
- Pádel
- Tenis

Para generar el informe se utilizan los datos de la tabla Reservas, junto con la relación hacia Instalaciones y TiposInstalacion.

El informe no utiliza filtros adicionales y trabaja sobre el histórico completo de reservas registradas.

Toda la lógica de agrupación y cálculo se realiza previamente en C# mediante LINQ. Las reservas se agrupan por tipo de instalación y para cada grupo se calculan las siguientes métricas:

- Número total de reservas
- Total de ingresos generados
- Media de ingresos por reserva

A diferencia de otros informes, Crystal Reports recibe directamente una estructura ya resumida y preparada, evitando cálculos complejos dentro del diseño .rpt.

La clase IngresosPorTipoInstalacionReportBuilder es la encargada de realizar toda esta agregación, cargar los datos en el DataSet tipado IngresosPorTipoInstalacionDataSet y devolver el informe listo para su visualización.

Informe Reservas por Fecha

El informe de reservas por fecha permite consultar toda la actividad del club dentro de un rango temporal concreto.

Es uno de los informes más útiles a nivel operativo, ya que facilita revisar reservas realizadas en períodos específicos, como:

- Día actual
- Semana en curso
- Mes actual
- Intervalos personalizados

El informe utiliza información de la tabla Reservas junto con los datos relacionados de Socios e Instalaciones.

Dispone de dos parámetros opcionales:

- fechaDesde

- fechaHasta

Si el usuario no introduce fechas, el sistema utiliza automáticamente el mes actual como rango predeterminado.

La fecha final se ajusta internamente hasta las 23:59:59 para incluir todas las reservas del último día seleccionado.

Las reservas filtradas se ordenan cronológicamente y se muestran en forma de listado detallado incluyendo:

- Nombre del socio
- Instalación reservada
- Fecha y hora de inicio
- Fecha y hora de fin
- Precio total
- Estado de la reserva

Crystal Reports puede mostrar adicionalmente un total global de ingresos al final del informe.

La construcción técnica se realiza mediante la clase `ReservasPorFechaReportBuilder`, que recibe las fechas como parámetros opcionales de tipo `DateTime?`, aplica los filtros utilizando LINQ y carga el resultado en el `DataSet` tipado `ReservasPorFechaDataSet`.

Finalmente, el visor `Window1` del proyecto `ProyectoFinal.ReportsView_new` recibe el tipo de informe mediante el enum `ReportType`, instancia automáticamente el Builder correspondiente y muestra el informe dentro del visor integrado de Crystal Reports para WPF.

9. Validaciones y control de errores

9.1 Validaciones centralizadas

Todas las validaciones se realizan desde los controladores mediante métodos privados específicos.

Esto garantiza coherencia y reutilización de lógica.

Además, las funciones de validación devuelven por referencia la información del error para poder mostrarla al usuario y aportar datos coherentes sobre el error que está ocurriendo. Por si fuera a causa de datos introducidos por el usuario o un error genérico del que se debe informar a los administradores de la aplicación para ser solucionado.

9.2 Ejemplo de Validaciones (reservas)

Las reservas validan:

- Existencia de socio e instalación.
- Horarios correctos.
- Duración mínima.
- Ausencia de solapamientos.
- Existencia de cuotas válidas.

9.3 Comunicación con el usuario

Todos los mensajes se muestran mediante ventanas modales personalizadas.

Existen tres tipos principales:

- Error.
- Información.
- Confirmación.

9.4 Gestión de excepciones

Las operaciones de base de datos están protegidas mediante bloques try-catch.

Las excepciones se gestionan en varios niveles para proporcionar mensajes claros al usuario.

10. Pruebas automatizadas

El proyecto incorpora pruebas automatizadas desarrolladas con MSTest.

Se incluyen:

- Tests unitarios.
- Tests de validación.
- Tests de integración con base de datos.

10.1 Tests de utilidades

Validan métodos auxiliares como:

- Validación de email.
- Validación de teléfono.

10.2 Tests de lógica de negocio

Comprueban validaciones importantes, como impedir reservas con fechas incorrectas.

10.3 Tests de integración

Verifican operaciones reales contra la base de datos, incluyendo inserción y recuperación de datos.

11. Problemas encontrados y soluciones

Durante el desarrollo surgieron distintos problemas técnicos que tuvieron que resolverse.

Problemas destacados

- El identificador de los socios no se actualizaba tras guardar.
- Errores de codificación en ficheros XAML.
- Dependencias faltantes en los tests.
- Necesidad de instalación del entity framework y la cadena de conexión en algunos proyecto
- Violaciones de arquitectura entre capas. A veces he usado directamente la api para acceder a los datos cuando debería accederse al controlador. Creo que todos están subsanados
- Problema con los reports, por mucho que aplicara un formato numérico sin comas me lo mostraba con comas, no he conseguido solucionarlo. Como nota, esto me pasa en el informe de socios activos, al mostrar el total en la cabecera
- Generación del instalable. Me daba muchos problemas el publicador base de visual studio clickonce, debido a la estructura de proyectos. Relacionado con el proyecto de crystal reports. He tenido que explorar otras maneras de crear el instalable. Tras hacer pruebas he optado por utilizar un instalador clásico MSI/Setup en lugar de ClickOnce porque la aplicación está compuesta por varios proyectos WPF, Crystal Reports y múltiples dependencias externas, lo que provocaba errores de validación

y conflictos de manifiestos durante la publicación automática con ClickOnce. Esto permite empaquetar todos los ejecutables, DLLs e informes necesarios en un único instalador más estable y compatible con aplicaciones de escritorio complejas como esta.

12. Seguridad

La aplicación no incorpora autenticación propia debido a que se utiliza en un entorno interno y controlado.

Aun así, se implementan medidas de protección como:

- Confirmaciones antes de borrar datos.
- Validaciones de integridad.
- Restricciones de negocio.
- Control de coherencia de reservas y cuotas.

13. Mejoras futuras

Entre las posibles mejoras futuras destacan:

- Sistema de login con roles.
- Exportación de informes a PDF.
- Notificaciones automáticas para avisar de las reservas.
- Estadísticas avanzadas con gráficos.
- Historial de pagos más detallado.
- Desarrollo de una API REST para futuras aplicaciones móviles.
- Mejoras para agilizar la creación masiva de cuotas

14. Conclusiones

El desarrollo de esta aplicación ha permitido aplicar de forma práctica múltiples conocimientos relacionados con el desarrollo de software de escritorio y la gestión de bases de datos.

Durante el proyecto se han utilizado tecnologías ampliamente utilizadas en entornos profesionales como:

- SQL Server.
- Entity Framework.

- WPF.
- Crystal Reports.
- Git y GitHub.

La arquitectura en capas implementada ha demostrado ser especialmente útil para mantener el código organizado, modular y fácil de mantener.

Además, la utilización de estilos globales, validaciones centralizadas y pruebas automatizadas ha permitido construir una aplicación sólida, coherente y preparada para futuras ampliaciones.

Anexos

A. Cadena de conexión

La cadena de conexión se encuentra definida en los ficheros App.config de los proyectos ejecutables bajo la clave:

ClubPolideportivoDBEntities

B. Configuración global del club

Los parámetros generales del sistema se almacenan en la clase ConfiguracionClub, del proyecto común. La idea es en un futuro extraer a la BBDD todas las posibles configuraciones y que se puedan modificar desde la propia aplicación

Entre ellos:

- Hora de apertura.
- Hora de cierre.
- Duración mínima de reservas.

C. Enum ReportType

Los informes disponibles se identifican mediante el enum:

ReportType

Valores disponibles:

- SociosActivos
- ReservasPorInstalacion

- IngresosPorTipoInstalacion
- ReservasPorFecha